

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de



Recap

- Vorstellungen Aufgaben
- Q&A

Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Kapitel 6: Test Driven Development



61. Motivation

62. Möglichkeiten von Tests

63. Test Driven Development

Unser Ziel

Bug-freie Software

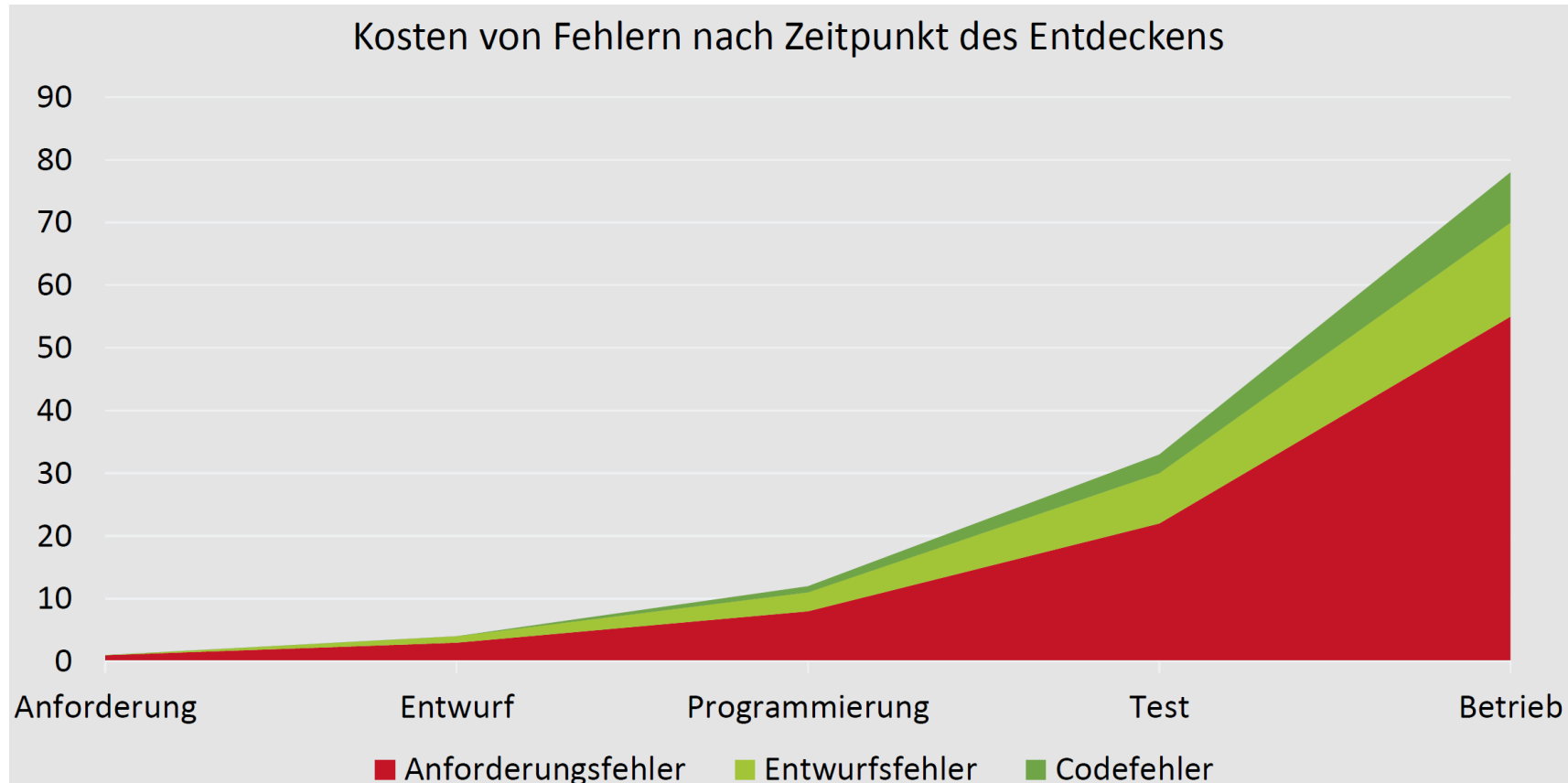
... gibt es aber nicht!

- Ab einer gewissen Komplexität gibt es keine Bug-freie Software
- Realistisches Ziel:
 - Auftreten von Bugs reduzieren
 - Sicherheit erhöhen, schwerwiegendere Bugs finden

Testen – Definition

- Ausführen eines Programms (oder eines Programnteils) um Fehler zu finden
 - *Kann Anwesenheit von Fehlern belegen, nicht Abwesenheit beweisen*
- Produktions-Software durchläuft vor Release eine Reihe von automatisierten Tests
 - *Beispiel: Amazon hat 2013 alle 11.6 Sekunden released, 2015 etwa jede Sekunde und mittlerweile öfter pro Sekunde*
 - *Dieser extreme Durchsatz wird durch das Prinzip Continuous Integration und Continuous Deployment (CI/CD) sowie eine strikte Microservices-Architektur ermöglicht.*
- Serienentwicklung Automotive: Ca Alle 1-3 Monate ein Serien-Release (Organisation in Sprint Release, mit jedem Commit ein Satz von zugehörigen Tests)

Warum sind frühzeitige Tests so wichtig?



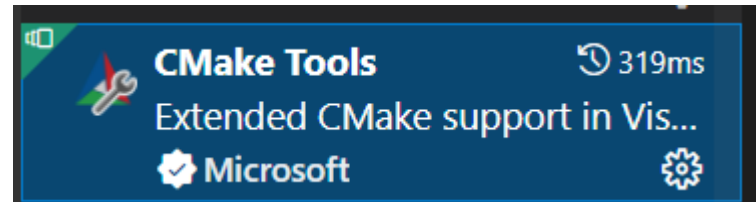
Automobilindustrie: Fehler können Menschenleben kosten

Motivation: Fehler finden!

Google Tests in VSCode über Cmake: Tools

- 1) Google Tests: [Download](#) (muss direkt in ihr Repository siehe nächste Slides) [User Guide](#), alternativ über FetchContent

- 2) VSCode Extension Cmake Tools

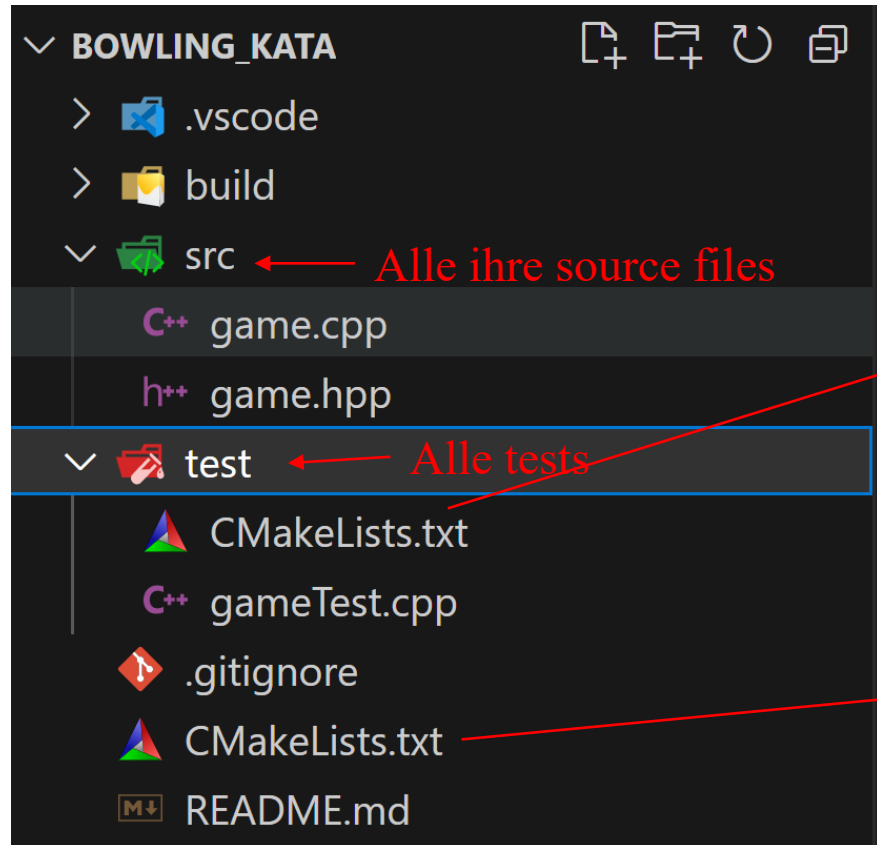


- 3) Cmake Programm: z.B. wenn Sie MSYS installiert haben für den Compiler (siehe set-up VSCode) können Sie eine MSYS Shell öffnen (default Speicherort C:\msys64\msys2.exe) und über diesen Befehl cmake installieren

`$pacman -S mingw-w64-x86_64-cmake`

- Youtube [Tutorial](#) über Gtests und Cmake

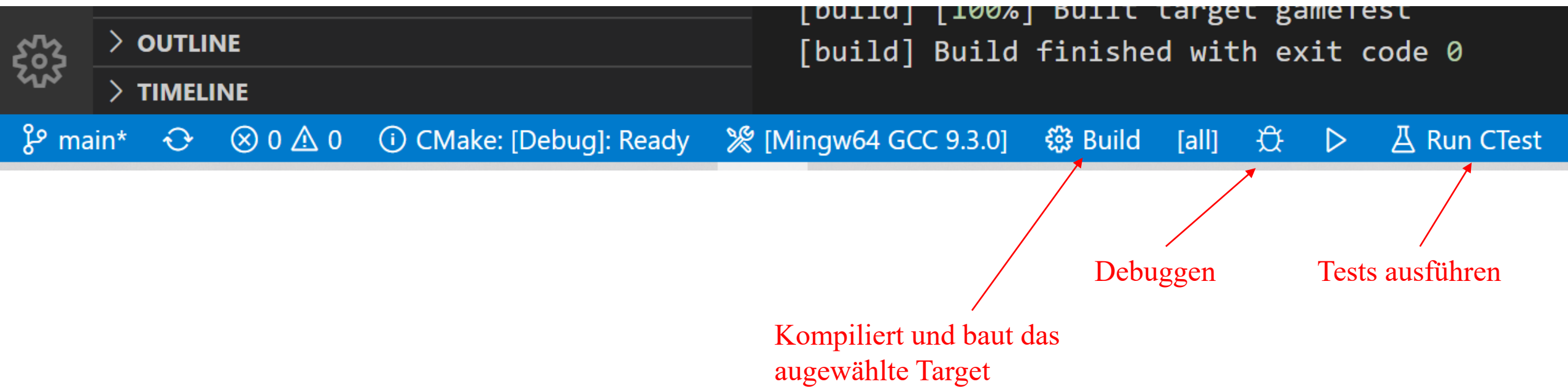
Google Tests in VSCode über Cmake: File-Set-up



```
CMakeLists.txt > ...
1  cmake_minimum_required(VERSION 3.8)
2
3  set(This bowling_kata)
4
5  project(${This})
6
7  set(CMAKE_CXX_STANDARD 11)
8
9  include(FetchContent)
10 FetchContent_Declare(
11     googletest
12     URL https://github.com/google/googletest/archive/refs/heads/main.zip
13 )
14 # For Windows: Prevent overriding the parent project's compiler/linker settings
15 set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
16 FetchContent_MakeAvailable(googletest)
17
18 enable_testing()
19
20 set(Headers
21     src/game.hpp
22 )
23
24 set(Sources
25     src/game.cpp
26 )
27
28 add_library(${This} STATIC ${Sources} ${Headers})
29 add_subdirectory(test)
30
```

```
m_drue04, 15 months ago | 1 author (m_drue04)
1  cmake_minimum_required(VERSION 3.8)
2
3  set(This gameTest)
4
5  set(Sources
6     gameTest.cpp)
7
8  add_executable(${This} ${Sources})
9
10 target_link_libraries(${This} PUBLIC
11     gtest_main
12     bowling_kata
13 )
14
15 add_test(
16     NAME ${This}
17     COMMAND ${This}
18 )
```

Google Tests in VSCode über Cmake: File-Set-up



Google Tests in VSCode über Cmake: Trouble Shooting (falls nicht automatisch konfiguriert)

- settings.json sollte beinhalten
Öffnen über str+shift+p

```
>opense
```

Preferences: **Open Settings (JSON)**

- cmake-tools-kits.json sollte beinhalten
Öffnen über str+shift+p

```
>
```

CMake: Edit User-Local CMake Kits

```
{ } settings.json x { } cmake-tools-kits.json
C: > Users > druep > AppData > Roaming > Code > User > { } settings.json > ...
25 "window.zoomLevel": 2,
26 // Cmake configuration
27 "cmake.cmakePath": "C:\\msys64\\mingw64\\bin\\cmake.exe",
28 "cmake.mingwSearchDirs": [
29   "C:\\msys64\\mingw64\\bin"
30 ],
31 "cmake.generator": "MinGW Makefiles"
32 }
```

```
settings.json { } cmake-tools-kits.json x
> Users > druep > AppData > Local > CMakeTools > { } cmake-tools-kits.j
1 [
2   {
3     "name": "Mingw64 GCC 9.3.0",
4     "compilers": {
5       "C": "C:\\msys64\\mingw64\\bin\\gcc.exe",
6       "CXX": "C:\\msys64\\mingw64\\bin\\g++.exe"
7     },
8     "preferredGenerator": {
9       "name": "MinGW Makefiles",
10      "platform": "x64"
11    },
12    "environmentVariables": {
13      "PATH": "C:/msys64/mingw64/bin/"
14    }
15  }
16 ]
```

Kapitel 6: Test Driven Development

61. Motivation



62. Möglichkeiten von Tests

63. Test Driven Development

Testdurchführung

Manuell

- Machen Sie für kleine Programme ständig → Schnell Ergebnisse
- Tests die nicht oder nur schlecht automatisiert werden können, z.B. User Acceptance Tests
- Ergebnisse müssen ebenfalls manuell dokumentiert werden
- Machen regelmäßige Releases aufwändig bis unmöglich

VS

Automatisiert

- Fast immer eingebunden in Entwicklungsprozess: z.B. Automatisch getriggert nach Git Commit
- Nachteil: erstes Testen bedeutend aufwändiger
- Dokumentation wird automatisch generiert: Test Katalog, Line und Branch Test-Abdeckung

Testarten (Auszug)

- **Unit Tests:**

- Unit ist eine Methode bzw. eine Funktion
- Mit gegebenem Input wird überprüft, ob z.B. die Funktion das erwartete Ergebnis liefert

```
TEST (Trigonometry test suite , Cos test)
{
    ASSERT_EQ (cos(0), 1);
    ASSERT_EQ (cos(1), 0.54030230586);
    ASSERT_EQ (cos(3.1415), -0.99999999);
}
```

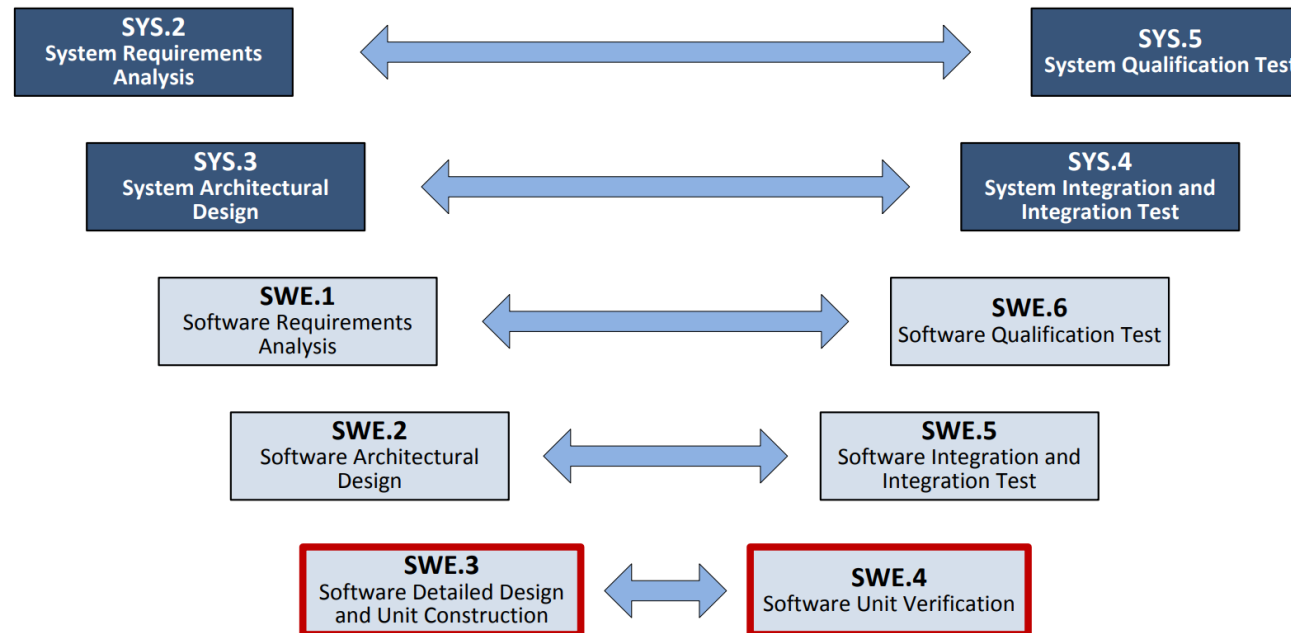
- **Komponenten Tests:**

- Testet Komponenten, z.B. eine Lib oder ein Runnable

- **Integrations Tests:**

- Testen mehrere Komponenten im Zusammenspiel

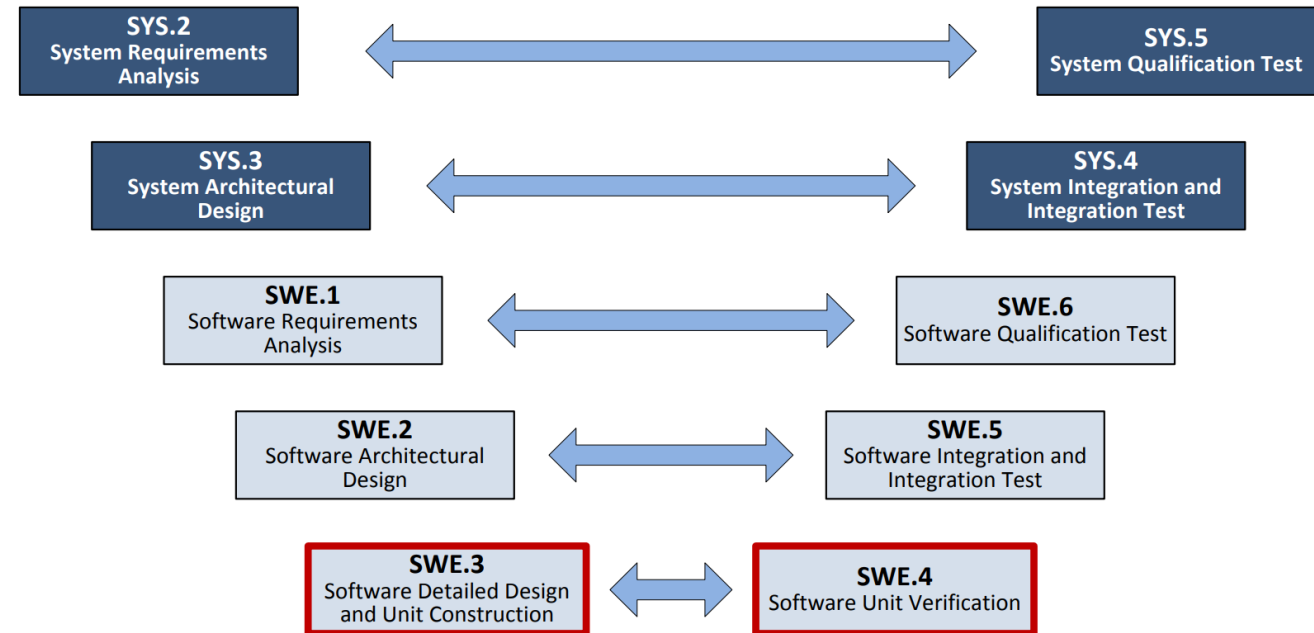
Testen nach ASPICE - Automotive Software Process Improvement and Capability Determination



Testen nach ASPICE - Automotive Software Process Improvement and Capability Determination

Linke Seite V-Modell

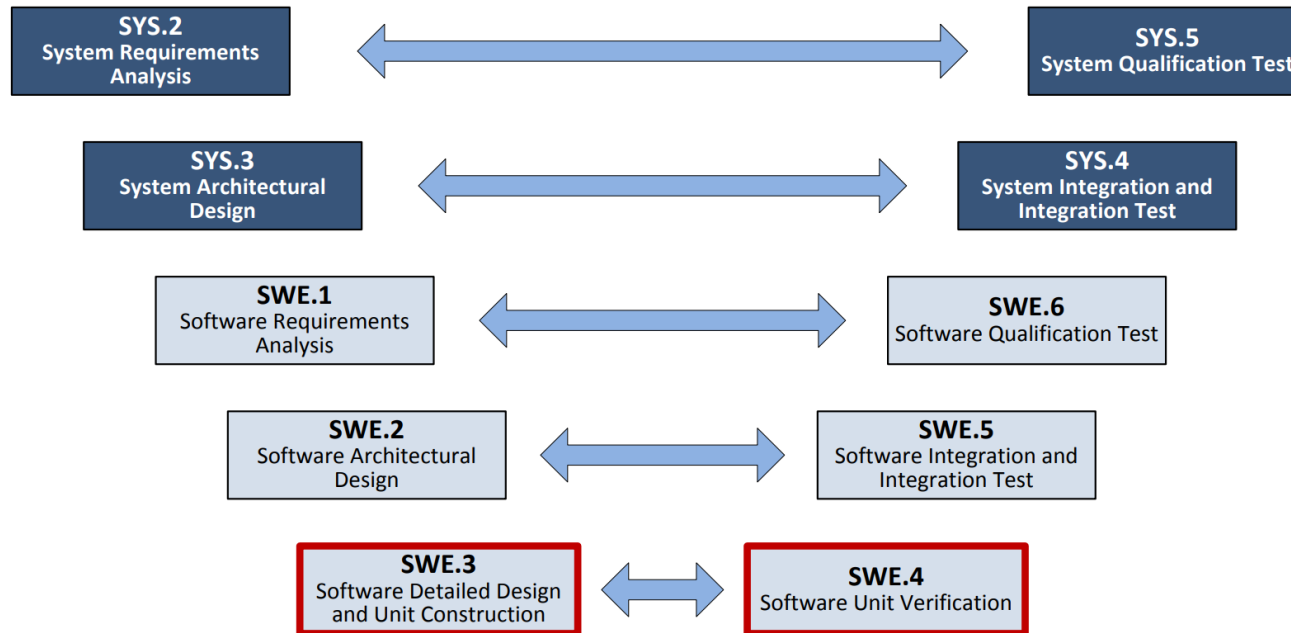
- Erst System Anforderungen (*When run conditions are met, the system shall follow the preceding vehicle in the defined distance.*)
- Dann Systemarchitektur (EE Architektur, Hardware Komponenten, Software-System-Komponenten wie Perception, Situationsanalyse, etc.)
- Dann Software Anforderungen: (*When run conditions are met the runnable xy shall deliver an estimation of ...*)
- Dann Software Architektur: Runnables und deren Verbindungen
- Dann Software Detailed Design: Spezifikation von Klassen, Methoden und Funktionen



Testen nach ASPICE - Automotive Software Process Improvement and Capability Determination

Rechte Seite V-Modell

- Unit Tests
- Software Integration Test
- Software Requirements Tests
- System Integrations und System Tests



Kapitel 6: Test Driven Development

61. Motivation

62. Möglichkeiten von Tests



63. Test Driven Development

Unit Tests

- Erwartung an Output wird definiert. Funktion wird aufgerufen. Output wird kontrolliert.

Given // **When** // **Then**

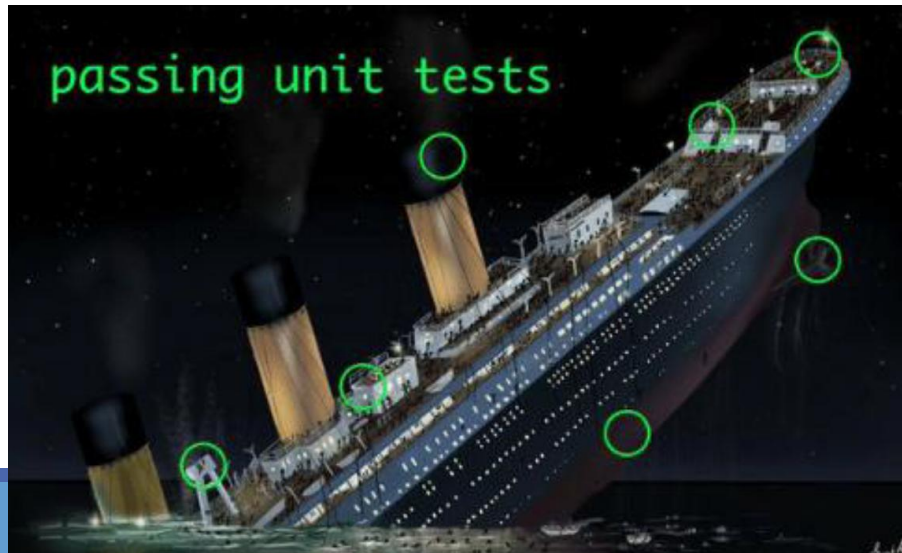
- Ziel: Auf keinen Fall Implementierung testen, sondern Verhalten
- Wie genau Implementierung ist nicht relevant und soll später auch noch verändert werden können: Wir wollen **Refactoring enablen**, nicht erschweren!
- Getestet werden soll deswegen immer eine API, d.h. alle (nicht trivialen) public Funktionen einer Klasse/einer Komponente
- Intern soll z.B. eine Methode in zwei, drei, vier private Funktionen aufgeteilt werden können. Das soll nicht erschwert werden.
- Testen von privaten Funktionen vermeiden!

Unit Tests

- Je besser die Kapselung einer Klasse, desto einfacher die Tests
- Je weniger Abhängigkeiten, desto einfacher der Test-Aufbau

Aber:

- Viele Unit Tests garantieren keine Sicherheit
- Unit Tests sollten durch System, Integrations und Komponententests ergänzt werden



Unit Tests

- Erwartung an Output wird definiert. Funktion wird aufgerufen. Output wird kontrolliert.

```
#include <gtest/gtest.h>
#include <cmath> // Für std::sqrt

// Testfall für die Quadratwurzelberechnung
TEST(SquareRootTest, PositiveNumber) {
    ASSERT_EQ(std::sqrt(25.0), 5.0); // Erwartet 5.0 für sqrt(25.0)
}

TEST(SquareRootTest, Zero) {
    ASSERT_EQ(std::sqrt(0.0), 0.0); // Erwartet 0.0 für sqrt(0.0)
}

TEST(SquareRootTest, NegativeNumber) {
    ASSERT_TRUE(std::isnan(std::sqrt(-25.0))); // Erwartet NaN für sqrt(-25.0)
}

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

Asserts and Expects

Overview of important Assert/Expects for the Googletest framework (see .md file on Github for more examples)

Basic Assertions

- `ASSERT_TRUE(condition)`: Fails the test if the condition is false.
- `ASSERT_FALSE(condition)`: Fails the test if the condition is true.
- `ASSERT_EQ(expected, actual)`: Fails the test if `expected` is not equal to `actual`.
- `ASSERT_NE(val1, val2)`: Fails the test if `val1` is equal to `val2`.
- `ASSERT_LT(val1, val2)`: Fails the test if `val1` is not less than `val2`.
- `ASSERT_LE(val1, val2)`: Fails the test if `val1` is not less than or equal to `val2`.
- `ASSERT_GT(val1, val2)`: Fails the test if `val1` is not greater than `val2`.
- `ASSERT_GE(val1, val2)`: Fails the test if `val1` is not greater than or equal to `val2`.

Aufgaben: Tests implementieren

1. Laden Sie sich den Ordner `IsPalindrom` herunter
2. Schreiben Sie Sinnvolle Unit-Tests in der Datei `test_main.cpp`.
3. Die Funktion `isPalindrom` soll getestet werden.

```
bool isPalindrome(const std::string& str) {  
    int left = 0;  
    int right = str.size() - 1;  
    while (left < right) {  
        if (str[left] != str[right]) return false;  
        left++;  
        right--;  
    }  
    return true;  
}
```


Unit Test

Testen Sie nicht die Implementierung!

```
bool isLeapYear(int year) {  
    if (year % 4 != 0) return false;  
    if (year % 100 != 0) return true;  
    return year % 400 == 0;  
}
```

Beispiel Test Implementierung

```
TEST(IsLeapYearTest, TestsImplementation) {  
    // Testet interne Details der Implementierung  
    // Prüft, ob die spezifischen Bedingungen erfüllt sind, anstatt das Verhalten der Funktion zu testen  
    EXPECT_TRUE(isLeapYear(4));    // Erreicht Zeile if (year % 100 != 0)  
    EXPECT_TRUE(isLeapYear(400));  // Erreicht return true  
    EXPECT_FALSE(isLeapYear(100)); // Erreicht return false  
    EXPECT_FALSE(isLeapYear(3));   // Erreicht return false  
}
```

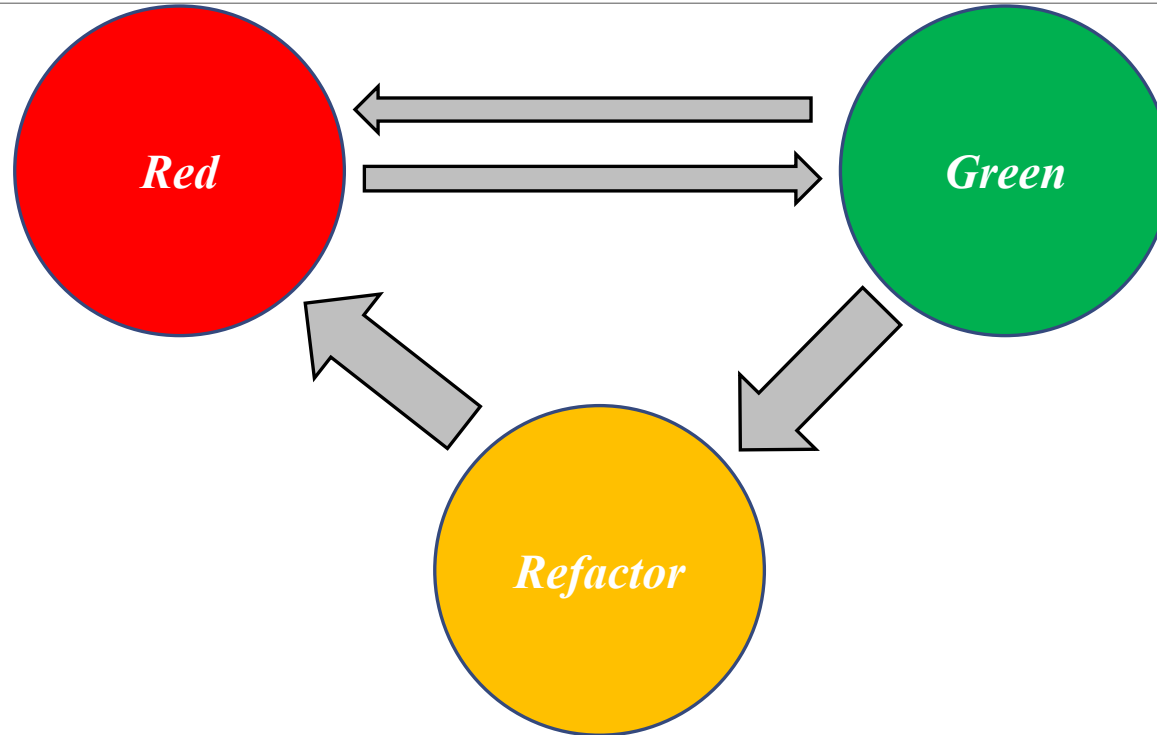
Beispiel Test Verhalten → Besser

```
TEST(IsLeapYearTest, HandlesTypicalCases) {  
    EXPECT_TRUE(isLeapYear(2000)); // Divisible by 400  
    EXPECT_FALSE(isLeapYear(1900)); // Divisible by 100 but not by 400  
    EXPECT_TRUE(isLeapYear(2004));  // Divisible by 4 but not by 100  
    EXPECT_FALSE(isLeapYear(2001)); // Not divisible by 4  
}  
  
TEST(IsLeapYearTest, HandlesEdgeCases) {  
    EXPECT_FALSE(isLeapYear(0));    // Edge case: year 0  
    EXPECT_FALSE(isLeapYear(-400)); // Negative year  
    EXPECT_TRUE(isLeapYear(1600));  // Historical leap year  
}
```

Gut testbarer Code

- Sollte I/O Code strikt von Business-Logik (also Algorithmik) trennen
- Sollte Abhängigkeiten minimieren (siehe Kapselung, Law of Demeter, ...)
- Funktionen die viele Variablen als Input haben (wenn es wirklich nicht möglich war diese zu refactorn) sollten selbst keine komplexen Algorithmen ausführen
- Funktionen mit komplexen Algorithmen sollten eine minimale Anzahl an Input Variablen besitzen
- Sollte keine globalen Variablen verwenden

Was ist Test Driven Development?



Die Regeln

- „Schreibe keinen Production Code ohne gefailten Unit Test.“
- „Refactore deinen Code erst nach gepasstem Unit Test.“

Ziel: Gut getesteter, gut designer Code

Google Tests Demo: Bowling Kata

1	4	4	5	6	▲	5	▲	▲	0	1	7	▲	6	▲	▲	2	▲	6
5		14		29		49		60		61		77		97		117		133

- Bowling Spiel besteht aus 10 Runden (*frames*).
- In jedem Frame können bis zu 10 Pins fallen.
- Punktzahl eines Frames: Anzahl der Pins + Mögliche Bonus Punkte von Spares und Strikes.
- Spare: Alle 2 Pins mit zwei Versuchen. Bonus: Nächste Wurf wird verdoppelt. Strike: Alle 10 Pins in einem Versuch. Bonus: Nächsten beiden Würfe werden verdoppelt.
- 10'ter Frame: Wenn ein Spare geworfen wird, gibt es noch einen extra Wurf. Wenn ein Strike geworfen wird, gibt es zwei extra Würfe. Beim Strike: Wenn beim ersten dieser Bonus Würfe wieder alle abgeräumt werden, werden die Pins für den letzten Wurf noch einmal aufgestellt.

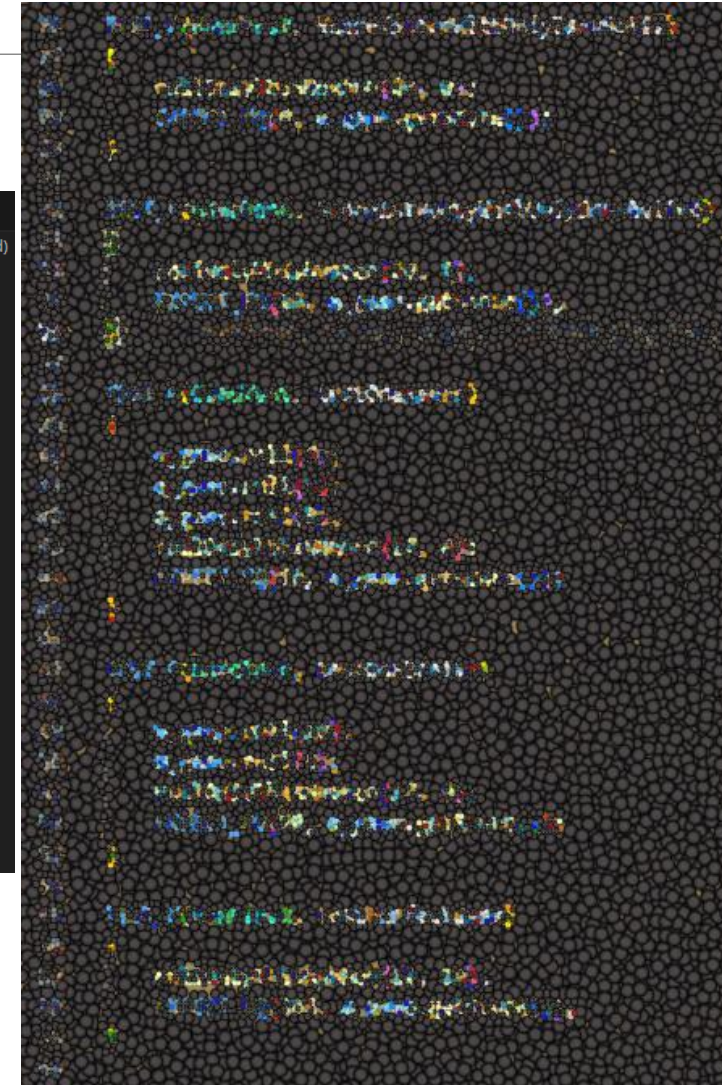
Aufgaben I: Bowling Kata in Google Tests

Anforderungen: API

Game
+ roll(pins : int) + score() : int

- Laden Sie Bowling Kata vom Repo
 - Implementieren Sie die Funktion getScore TestDriven.
 - Score kann nur am Ende des Spiels, also nach 10 Frames gecallt werden.
- 1) Bringen Sie Google Tests in ihrer Entwicklungsumgebung zum Laufen.
 - 2) Implementieren Sie den Rest der Tests. Einen nach dem anderen.
Red → Green → Refactor

```
gameTest.cpp x
test > C++ gameTest.cpp > TEST_F(GameTest, ScoresTwentyWithOnlyOnesRolled)
You, 5 minutes ago | 2 authors (m_drue04 and others)
1 #include "gtest/gtest.h"
2 #include "../src/game.hpp"
3
4
5 You, 3 months ago | 2 authors (m_drue04 and others)
6 class GameTest
7 : public ::testing::Test
8 {
9 protected:
10     Game m_game;
11
12     void SetUp() override
13     {
14         m_game = Game();
15     }
16
17     void rollOnlyThisNumber(int nRolls, int rollResult)
18     {
19         for (int i = 0; i < nRolls; ++i)
20         {
21             m_game.roll(rollResult);
22         }
23     }
24 }
```



Mocken

- Manchmal hat man Abhängigkeiten, die Unumgänglich oder aufwändig aufzubauen sind
- Möchte man eine Klasse/Komponente/.. Testen, die eine derartige Abhängigkeit hat, so „mockt“ man die Abhängigkeit
- Man erstellt ein Mock Objekt und definiert dessen Verhalten i.d.R. geschieht das über das Überschreiben der public Methoden mithilfe einfacher Rückgabewerte
- Verschiedene Frameworks erledigen den Großteil der Arbeit für uns (hier z.B. Google Mock)
- In C++ müssen die Methoden die überschrieben, d.h. gemockt, werden dafür i.d.R. virtual sein

Mocken

Was für Objekte wollen wir mocken?

- Komplex im Aufbau z.B. Datenbanken müssten jedes Mal initial befüllt werden → wäre viel Aufwand
- Klassen, deren Verhalten sich verändern kann relevant sind für uns die Werte, die rauskommen → der Test müsste sonst häufig angepasst werden
- Objekte mit nicht deterministischen Funktionen → wie können wir das Ergebnis unserer Funktion prüfen, wenn wir nicht wissen, welche Werte sie von dem Objekt bekommt?
- Schwierig reproduzierbare Zustände Exceptions , Netzwerkfehler, etc. die i.d.R. nicht auftreten sollen, die man aber im Notfall handeln will.

Mocken

```
1 #include <gtest/gtest.h>
2 #include <gmock/gmock.h>
3 #include "../greeter_service.h"
4 #include "../time_provider.h"
5
6 class MockTimeProvider : public TimeProvider {
7 public:
8     // Define the mock method for getCurrentHour
9     // Arguments: MOCK_METHOD(ReturnType, MethodName, (ArgumentTypes), (Qualifiers))
10    MOCK_METHOD(int, getCurrentHour, (), (const, override));
11 };
12
13 class GreeterServiceTest : public ::testing::Test {
14 protected:
15     MockTimeProvider mockTimeProvider;
16     GreeterService greeter{&mockTimeProvider};
17 };
18
19 TEST_F(GreeterServiceTest, GreetInMorning_ReturnsGoodMorning) {
20     EXPECT_CALL(mockTimeProvider, getCurrentHour())
21         .WillOnce(::testing::Return(8));
22     std::string result = greeter.greet("Alice");
23     EXPECT_EQ(result, "Good morning, Alice!");
24 }
25
26 TEST_F(GreeterServiceTest, GreetAtNoon_ReturnsGoodAfternoon) {
27     EXPECT_CALL(mockTimeProvider, getCurrentHour())
28         .WillOnce(::testing::Return(12));
29     std::string result = greeter.greet("Bob");
30     EXPECT_EQ(result, "Good afternoon, Bob!");
31 }
32
33 TEST_F(GreeterServiceTest, GreetInAfternoon_ReturnsGoodAfternoon) {
34     EXPECT_CALL(mockTimeProvider, getCurrentHour())
35         .WillOnce(::testing::Return(15));
36     std::string result = greeter.greet("Charlie");
37     EXPECT_EQ(result, "Good afternoon, Charlie!");
38 }
```


Aufgaben II: Übungsprojekt Unit Tests



- 1) Integrieren Sie Google Tests in ihre 4-Gewinnt Implementierung und schreiben Sie Tests für die public Funktionen ihrer Klassen. Dafür kann und sollte refactort werden.
- 2) Entwickeln Sie Test-driven einen intelligenten Bot. Schreiben Sie hierfür Tests in denen Sie bei gegebenem Board-State bestimmte Spielzüge des Bots erwarten.